KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 3
상속과
클래스 변수

틀에 새긴 값과
물려받는 틀

```
class Bread:
    shop = "붕어빵 1호점"

class CreamBread(Bread):
    def __init__(self, f, p, cream):
        super().__init__(f, p)
        self.cream = cream
```

오늘의 순서

1	왜 필요한가	복사해서 만든 슈크림 틀, 틀에 새긴 값과 물려받는 틀	4–5쪽
2	클래스 변수	틀에 적은 값, 한 번에 바꾸기, 개수 세기, 두 변수 비교	6–9쪽
3	상속	물려받기, 더하기, super().__init__, 덮어쓰기, 섞어 담기, 눌러 보기	10–17쪽
4	복사와 상속 비교	고칠 곳 한 군데, 한 종류일 때만	18–19쪽
5	오류와 함정	super 빠짐, self 두 번, 재료 수, a.shop, self.count	20–24쪽
6	실습	도장 카드와 VIP 카드, 붕어빵과 미니 붕어빵 메뉴판	25–28쪽

3쪽은 노트북 준비, 29–30쪽은 막힐 때와 한 장 요약이에요.

들어가기

25에서 이어서 해요

`class, __init__, self`	24
메서드, a.intro()	24
`__str__, __repr__`	25
지역 변수와 전역 변수	20
for, 컴프리헨션	13, 23

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

슈크림 붕어빵 틀을 새로 만들면 거의 다 복사예요

[1]
```
class CreamBread:
    def __init__(self, flavor, price, cream):
        self.flavor = flavor
        self.price = price
        self.shop = "붕어빵 1호점"
        self.cream = cream

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

c = CreamBread("슈크림", 1200, "많이")
c.intro()
```

Bread에서 그대로 복사

← 가게 이름도 붕어빵마다 따로

← 다른 건 이 한 줄뿐

Bread에서 그대로 복사

```
슈크림 붕어빵 1200 원
```

소개 문구를 고치려면 두 틀을 다 고쳐야 해요.

틀에 새기는 값, 그리고 틀을 물려받는 틀

틀 class Bread

```
shop = "붕어빵 1호점"   ← 클래스 변수
__init__(flavor, price)
intro()
```

a	Bread
flavor	"팥"
price	1000

b	Bread
flavor	"피자"
price	1500

물려받기 = 상속

틀 class CreamBread(Bread)

```
cream
나머지는 물려받아요
```

c	CreamBread
flavor	"슈크림"
price	1200
cream	"많이"

c도 intro와
shop이 있어요

2. 클래스 변수

틀에 바로 적은 값은 모든 붕어빵이 같이 봐요

[1]
```
class Bread:
    shop = "붕어빵 1호점"    ← 틀에 바로 적기

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.shop)
print(b.shop)
print(Bread.shop)
```

```
붕어빵 1호점
붕어빵 1호점
붕어빵 1호점
```
셋 다 틀의 값



① a 안에서 shop 찾기 ✕ 없어요

② a를 찍은 틀 Bread에서 찾기 ✓ 붕어빵 1호점

2. 클래스 변수

틀의 값을 바꾸면 모든 붕어빵에 한 번에 바뀌어요

```
[2] Bread.shop = "붕어빵 2호점"    ← 틀 이름으로 바꿔요
    print(a.shop)
    print(b.shop)
```

```
붕어빵 2호점
붕어빵 2호점
```

틀 `class Bread`

`shop = "붕어빵 2호점"`



붕어빵이 100개여도 Bread.shop 한 줄이면 돼요.

2. 클래스 변수

찍을 때마다 틀에 개수를 세요

[3]
```
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        Bread.count = Bread.count + 1 ← 틀에 +1

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
c = Bread("피자", 1500)
print(Bread.count)
```

```
3
```

Bread.count

3

`a = Bread("팥", 1000)`	+1 → 1
`b = Bread("슈크림", 1200)`	+1 → 2
`c = Bread("피자", 1500)`	+1 → 3

가게 이름, 전체 개수처럼 틀 전체에 하나만 있으면 되는 값에 써요.

2. 클래스 변수

클래스 변수와 인스턴스 변수

	인스턴스 변수	클래스 변수
어디에 적나요	`__init__` 안, `self.flavor = ...`	`class` 바로 아래, `shop = ...`
누구 것인가요	붕어빵 하나하나 (객체)	틀 하나 (클래스)
몇 개 있나요	붕어빵 수만큼	틀에 하나
꺼내기	`a.flavor`	`a.shop`, `Bread.shop`
바꾸기	`a.flavor = ...`	`Bread.shop = ...`
예	맛, 가격, 도장 수	가게 이름, 전체 개수

인스턴스는 객체와 같은 말이에요. 붕어빵마다 다르면 self.이름, 모두 같으면 틀에 적어요.

3. 상속

괄호에 부모 틀을 적으면 전부 물려받아요

[1]
```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

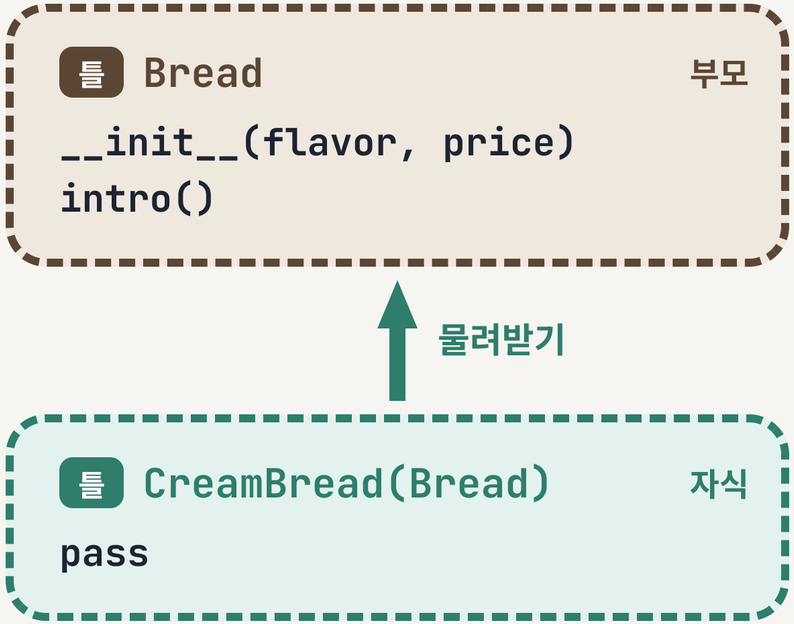
    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

class CreamBread(Bread):  ← 괄호에 부모 틀
    pass

c = CreamBread("슈크림", 1200)
c.intro()
print(c.flavor)
```

```
슈크림 붕어빵 1200 원
슈크림
```

pass뿐인데 intro도, flavor도 있어요



c	CreamBread
flavor	"슈크림"
price	1200

3. 상속

자식 틀에는 더 할 일을 적어요

[2]
```
class CreamBread(Bread):
    def keep(self):
        print(self.flavor, "붕어빵은 차갑게 보관해요")

a = Bread("팥", 1000)
c = CreamBread("슈크림", 1200)
c.intro()
c.keep()
```

자식 틀에만 있는 메서드

```
슈크림 붕어빵 1200 원
슈크림 붕어빵은 차갑게 보관해요
```

← intro는 부모에게, keep은 자기 틀에서

[3]
```
a.keep()
```

```
AttributeError: 'Bread' object has no attribute 'keep'
```

← a는 부모 틀 Bread로 찍었어요

물려받기는 부모에서 자식 쪽으로만 일어나요.

3. 상속

칸을 더할 땐 super().__init__으로 부모에게 맡겨요

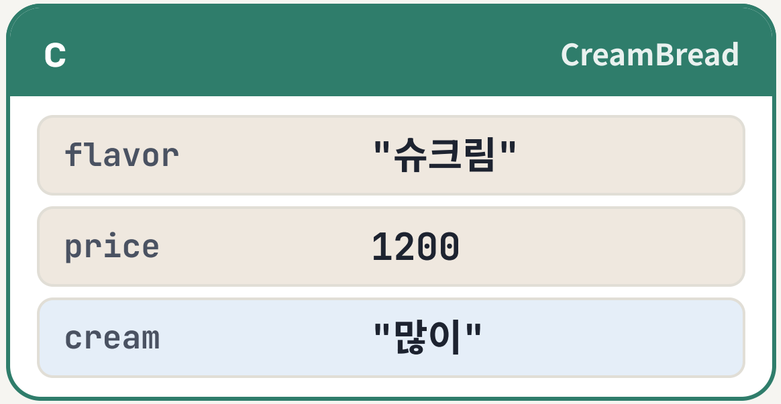
[4]
```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream  ← 자식만 가진 칸은 직접 붙여요

c = CreamBread("슈크림", 1200, "많이")
print(c.flavor, c.price, c.cream)
```

```
슈크림 1200 많이
```

`super()`.`__init__(flavor, price)`

- `super()` — 부모 틀
- `__init__(flavor, price)` — 부모의 __init__에 맛과 가격을 넣어요. self는 안 적어요



맛과 가격은 부모가, 크림은 자식이 붙여요.

3. 상속

CreamBread("슈크림", 1200, "많이") 한 줄에서 일어나는 일

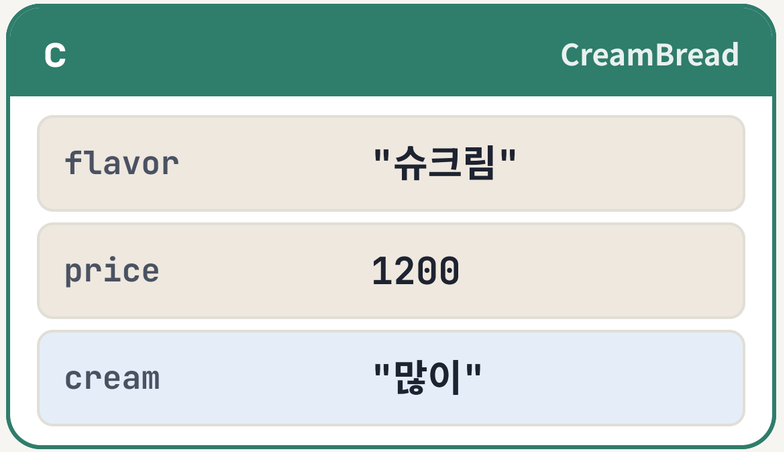
① **CreamBread.__init__**
자식의 __init__이 불려요. self는 새 붕어빵

② **super().__init__(flavor, price)**
부모의 __init__으로 가요. 같은 self예요

③ **self.flavor, self.price**
부모가 맛과 가격 칸을 붙여요

④ **self.cream = cream**
자식으로 돌아와 크림 칸을 붙여요

⑤ **c = 다 된 붕어빵**
세 칸이 다 붙은 붕어빵이 c에 담겨요



부모의 __init__에서도 자식의 __init__에서도 같은 self예요

3. 상속

같은 이름의 메서드를 다시 적으면 자식 것이 쓰여요

[5]
```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

    def intro(self):  ← 부모에도 있는 이름으로 다시 적기 = 덮어쓰기
        print(self.flavor, "붕어빵", self.price, "원, 크림", self.cream)

a = Bread("팥", 1000)
c = CreamBread("슈크림", 1200, "많이")
a.intro()
c.intro()
```

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원, 크림 많이
```

← a.intro(): Bread의 intro

← c.intro(): CreamBread의 intro

메서드는 자식 틀에서 먼저 찾고, 없으면 부모 틀로 올라가요.

3. 상속

부모 것을 부르고 조금만 더할 수도 있어요

[6]
```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

    def intro(self):
        super().intro()                    ← 부모의 intro를 그대로 불러요
        print("  크림:", self.cream)       ← 한 줄만 더해요

c = CreamBread("슈크림", 1200, "많이")
c.intro()
```

```
슈크림 붕어빵 1200 원    ← 부모 intro가 찍은 줄
  크림: 많이             ← 자식이 더한 줄
```

__init__	`super().__init__(flavor, price)`	부모의 __init__ 부르기
intro	`super().intro()`	부모의 intro 부르기

3. 상속

섞어 담아도 붕어빵마다 자기 intro를 불러요

[7]
```
order = [Bread("팥", 1000), CreamBread("슈크림", 1200, "보통"), Bread("피자", 1500)]
for b in order:
    b.intro()  ← 부르는 줄은 하나
```

```
팥 붕어빵 1000 원          ← Bread의 intro
슈크림 붕어빵 1200 원      ┐
  크림: 보통               ┘ CreamBread의 intro
피자 붕어빵 1500 원        ← Bread의 intro
```

[8]
```
print(isinstance(c, CreamBread))   ← isinstance(값, 틀): 그 틀로 찍었나요?
print(isinstance(c, Bread))
print(isinstance(a, CreamBread))
```

```
True    ← c는 CreamBread로 찍었어요
True    ← 슈크림 붕어빵도 붕어빵이에요
False   ← a는 CreamBread가 아니에요
```

3. 상속

눌러 보기: 파이썬은 어디서부터 찾을까요

틀 class Bread

```
shop = "붕어빵 1호점"
__init__(self, flavor, price)
intro(self)
```

물려받기

틀 class CreamBread(Bread)

```
__init__(self, flavor, price, cream)
intro(self)  # super().intro() + 크림
keep(self)
```

a — Bread

```
flavor  "팥"
price   1000
```

Bread 틀로 찍었어요

c — CreamBread

```
flavor  "슈크림"
price   1200
cream   "많이"
```

CreamBread 틀로 찍었어요

누구에게 [a] [c]

무엇을 [.intro()] [.keep()] [.shop] [.cream] [.flavor]

찾아보기 ▶ `c.intro()`

찾는 길이 여기에 한 줄씩 나와요.

출력

누구에게 무엇을 물을지 고르고 찾아보기를 눌러요. 브라우저로 슬라이드를 열면 눌러져요.

4. 복사와 상속 비교

복사해서 만든 틀과 물려받은 틀

복사 (1파트)

```
class CreamBread:
    def __init__(self, flavor, price, cream):
        self.flavor = flavor  ← 다시 적음
        self.price = price
        self.cream = cream

    def intro(self):
        print(self.flavor, ...)  ← 다시 적음
```

상속 (3파트)

```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

    def intro(self):
        super().intro()  ← 부모에게 맡김
        print("  크림:", self.cream)
```

	복사	상속
부모를 고치면	두 군데 다 고쳐야 해요	자식도 같이 바뀌어요
적는 것	모든 줄	다른 점만

4. 복사와 상속 비교

"~는 ~의 한 종류"일 때 상속해요

✓ 상속	슈크림 붕어빵은 붕어빵의 한 종류예요	`CreamBread(Bread)`
✓ 상속	미니 붕어빵은 붕어빵의 한 종류예요	`MiniBread(Bread)`
✓ 상속	VIP 카드는 도장 카드의 한 종류예요	`VipCard(Card)`
× 따로	붕어빵 노점은 붕어빵의 한 종류가 아니에요	Stall 틀을 따로 (25)
× 따로	음료는 붕어빵의 한 종류가 아니에요	Drink 틀을 따로

코드가 비슷해 보여도 한 종류가 아니면 상속하지 않아요.

super().__init__을 빼먹으면 부모 칸이 없어요

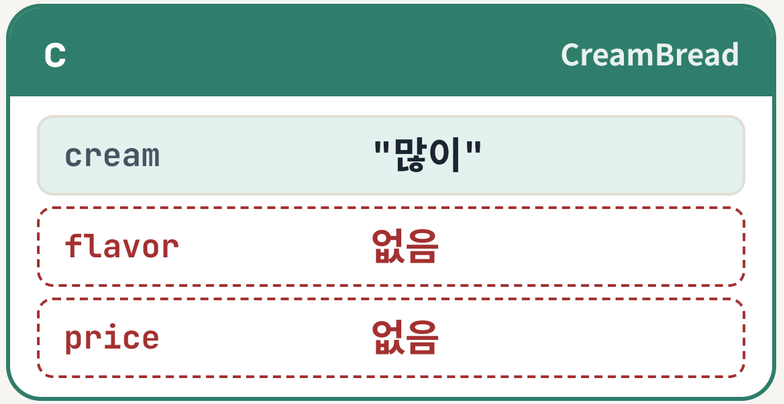
[1]
```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        self.cream = cream  ← super().__init__ 줄이 없어요

c = CreamBread("슈크림", 1200, "많이")
c.intro()
```

```
AttributeError: 'CreamBread' object has no attribute 'flavor'
```



자식 __init__ 첫 줄에 적어요

✓ `super().__init__(flavor, price)`

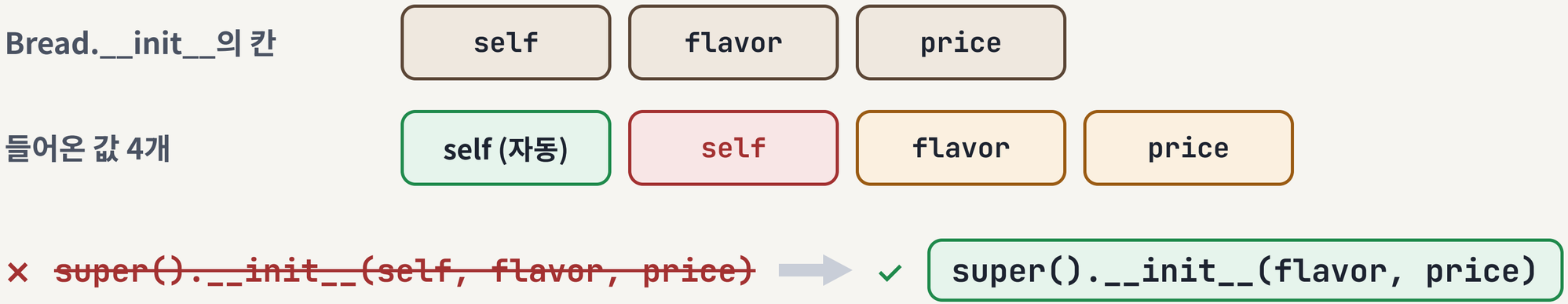
5. 오류와 함정

super().__init__에 self를 넣으면 4 were given

[2]
```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(self, flavor, price)  ← self를 직접 넣었어요
        self.cream = cream

c = CreamBread("슈크림", 1200, "많이")
```

```
TypeError: Bread.__init__() takes 3 positional arguments but 4 were given
```



× ~~super().__init__(self, flavor, price)~~ → ✓ `super().__init__(flavor, price)`

5. 오류와 함정

자식 틀은 재료가 하나 더 필요해요

[3]
```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

c = CreamBread("슈크림", 1200)   ← 재료를 2개만 넣었어요
```

```
TypeError: CreamBread.__init__() missing 1 required positional argument: 'cream'
```

CreamBread.__init__의 칸	self	flavor	price	cream
들어온 값	self (자동)	"슈크림"	1200	없음

× ~~CreamBread("슈크림", 1200)~~ → ✓ `CreamBread("슈크림", 1200, "많이")`

크림을 자주 안 적으면 20에서 배운 기본값으로 cream="보통"을 둘 수도 있어요.

a.shop = 으로 바꾸면 a에만 새 칸이 생겨요

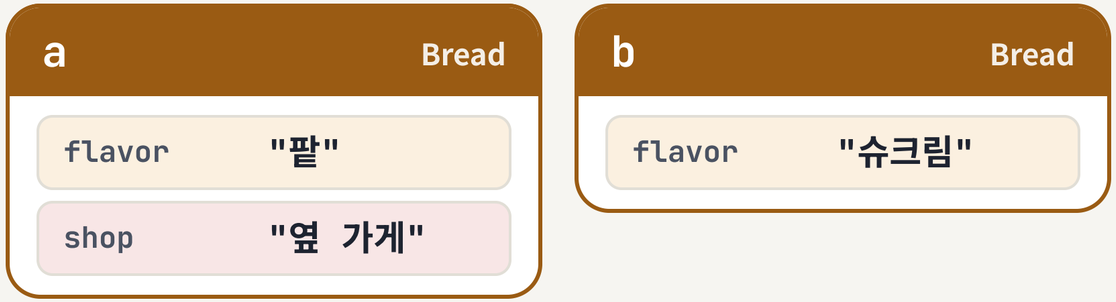
[4]
```
class Bread:
    shop = "붕어빵 1호점"

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.shop = "옆 가게"   ← 붕어빵 이름으로 바꾸기
print(a.shop)
print(b.shop)
print(Bread.shop)
```

옆 가게 ← a에 붙은 칸이 먼저 보여요
붕어빵 1호점
붕어빵 1호점] b와 틀은 그대로

틀 class Bread
shop = "붕어빵 1호점" ← 그대로예요



모두 바꾸려면 틀 이름으로

× ~~a.shop = "옆 가게"~~

✓ Bread.shop = "옆 가게"

self.count로 세면 틀의 개수가 안 늘어요

[5]
```
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        self.count = self.count + 1

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.count, b.count)
print(Bread.count)
```

```
1 1
0
```

← a.count, b.count

← Bread.count

```
self.count = self.count + 1
```

왼쪽: 붕어빵에 새 칸을 붙여요

오른쪽: 붕어빵에 없으니 틀의 0을 읽어요

틀 class Bread

count = 0 ← 한 번도 안 바뀌어요

a	Bread
flavor	"팥"
count	1

b	Bread
flavor	"슈크림"
count	1

× ~~self.count = self.count + 1~~ → ✓ `Bread.count = Bread.count + 1`

6. 실습

도장 카드 ①: 틀에 가게와 목표, VIP는 두 배로

[1]

```
class Card:
    shop = "KANT 카페"   ← 모든 카드가 같은 값: 클래스 변수
    goal = 10

    def __init__(self, name):
        self.name = name
        self.stamps = 0

    def add(self):
        self.stamps = self.stamps + 1

    def __str__(self):   ← 남은 도장 수까지 글자로 (25)
        left = Card.goal - self.stamps
        return f"[{Card.shop}] {self.name}: 도장 {self.stamps}개, {left}개 남음"

class VipCard(Card):   ← 도장 카드의 한 종류, add만 두 개씩
    def add(self):
        self.stamps = self.stamps + 2
```

출력 없이 틀만 만드는 셀이에요

6. 실습

도장 카드 ②: 같은 add인데 카드마다 다르게

```
[2] kim = Card("kim")
    lee = VipCard("lee")
    kim.add()
    kim.add()
    lee.add()
    lee.add()
    print(kim)
    print(lee)
```

```
[KANT 카페] kim: 도장 2개, 8개 남음
[KANT 카페] lee: 도장 4개, 6개 남음
```

kim — Card

name	"kim"
stamps	2

lee — VipCard

name	"lee"
stamps	4

`kim.add()` → `Card.add` +1씩

`lee.add()` → `VipCard.add` +2씩

print와 가게 이름은 둘 다 Card에서 물려받아요

6. 실습

붕어빵 메뉴판 ①: 미니 붕어빵 틀을 물려받기

[3]

```
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        Bread.count = Bread.count + 1  ← 찍을 때마다 개수 세기

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

class MiniBread(Bread):  ← 미니 붕어빵도 붕어빵의 한 종류
    def __init__(self, flavor, price, qty):
        super().__init__(flavor, price)  ← 맛과 가격은 부모에게 맡겨요
        self.qty = qty

    def __str__(self):  ← 묶음 가격이 보이게 덮어쓰기
        return f"미니 {self.flavor} {self.qty}개 {self.price * self.qty:,}원"
```

이 셀도 출력이 없어요

6. 실습

붕어빵 메뉴판 ②: 섞어 담고 찍고 세기

[4]
```
menu = [Bread("팥", 1000), MiniBread("팥", 300, 5), Bread("슈크림", 1200), MiniBread("슈크림", 400, 5)]
for m in menu:
    print(m)
print("찍은 붕어빵:", Bread.count)
print([m.flavor for m in menu if isinstance(m, MiniBread)])
```

```
팥 붕어빵 1,000원
미니 팥 5개 1,500원
슈크림 붕어빵 1,200원
미니 슈크림 5개 2,000원
찍은 붕어빵: 4
['팥', '슈크림']
```

붕어빵마다 자기 틀의 __str__ (25)

← 미니도 부모 __init__을 거쳐서 4개

← isinstance와 컴프리헨션(23)으로 미니만

24부터 26까지 배운 클래스를 한 셀에서 다 썼어요.

정리

막히면 오류의 마지막 줄을 봐요

1	`AttributeError: 'CreamBread' object has no attribute 'flavor'` 자식 __init__에서 super().__init__을 안 불렀어요	`super().__init__(...)`
2	`TypeError: ... takes 3 positional arguments but 4 were given` super().__init__에 self를 넣었어요	self 빼기
3	`TypeError: CreamBread.__init__() missing ... 'cream'` 자식 틀의 재료가 모자라요	칸 수 맞추기, 기본값
4	`NameError: name 'Bred' is not defined` 괄호 안 부모 이름 오타이거나 부모를 아직 안 만들었어요	부모 셀 먼저 실행
5	**한 붕어빵만 값이 달라요** a.shop = 으로 바꿨어요	`Bread.shop =`
6	**개수가 0에서 안 늘어요** self.count로 셌어요	`Bread.count`
7	**부모 메서드를 고쳤는데 자식은 그대로예요** 자식이 같은 이름으로 덮어썼어요	super().메서드()

헷갈리면 "자식 틀에서 먼저 찾고, 없으면 부모 틀" 순서로 따라가 봐요.

한 장 요약

`shop = "1호점"`
틀에 새긴 값 (클래스 변수)

`Bread.shop = ...`
모두 한 번에 바꾸기

`class A(Bread):`
틀 물려받기

`super().__init__(...)`
칸을 더할 때 부모에게 맡기기

`def intro(self):`
같은 이름이면 덮어쓰기

`super().intro()`
부모 것을 부르고 조금 더

`isinstance(c, Bread)`
어떤 틀로 찍었나

~는 ~의 한 종류
상속할지 고르는 말

붕어빵마다 다르면 self.이름, 모두 같으면 틀에 적어요. 한 종류면 물려받아요.

클래스 세 챕터 끝: 붕어빵 틀로 데이터와 할 일을 한 곳에 묶었어요.